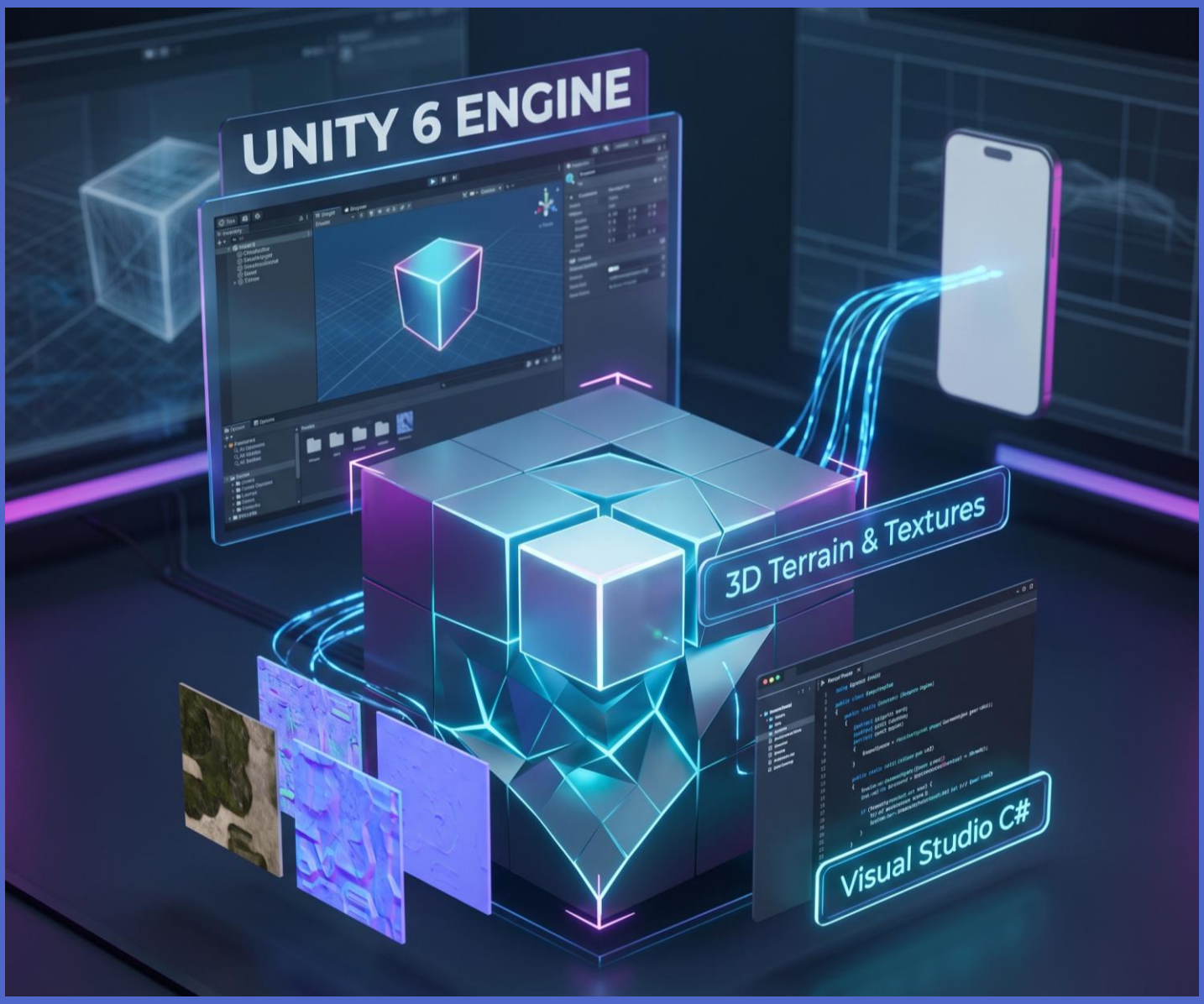
# PROGRAMACION MULTIMEDIA

## CASO PRACTICO I UD4




ALUMNO CESUR 25/26

Alejandro Muñoz de la Sierra

PROFESOR

Gonzalo Pérez Crespo

# I N T R O D U C C I O N

Al llegar a esta última etapa del ciclo, nos metemos de lleno en la Unidad 4, que va puramente de desarrollo de videojuegos en 2D y 3D. Para nosotros, esto es un poco el punto fuerte de Programación Multimedia y Dispositivos Móviles, porque nos saca de la rutina de hacer las típicas aplicaciones de gestión más rígidas y nos deja trastear por fin con motores gráficos actuales. En este caso práctico no queríamos hacer lo mínimo para aprobar y pasar el trámite; la idea era montar un entorno de trabajo en condiciones desde el minuto uno. Por eso, decidimos instalar Unity 6 (en concreto la versión 6000.4.4f1) y enlazarlo con el IDE Visual Studio Community 2026. Así nos asegurábamos de estar usando las mismas herramientas y versiones que se piden ahora mismo en cualquier estudio de desarrollo.

El objetivo principal de la tarea era crear un terreno que midiera exactamente 30x10 unidades. Después de darle un par de vueltas a lo que ofrece Unity, descartamos usar el *Terrain Engine* que viene por defecto. Esa herramienta está genial para generar mundos abiertos enormes, pero para una plataforma tan pequeña nos parecía un derroche de recursos brutal. Optamos por tirar de una primitiva matemática básica: un simple cubo escalado. Buscábamos clavar las medidas exactas tocando el componente *Transform* y demostrar que sabemos optimizar el motor sin usar herramientas pesadas si no hacen falta.

En esta memoria vamos a documentar cómo hemos pasado de tener un bloque geométrico gris a un suelo rocoso bastante realista. Explicaremos cómo nos hemos peleado con conceptos de renderizado físico que hasta ahora ni nos sonaban, y cómo hemos integrado recursos de fuera para que el proyecto luzca bastante mejor que usando los *assets* que vienen de serie.

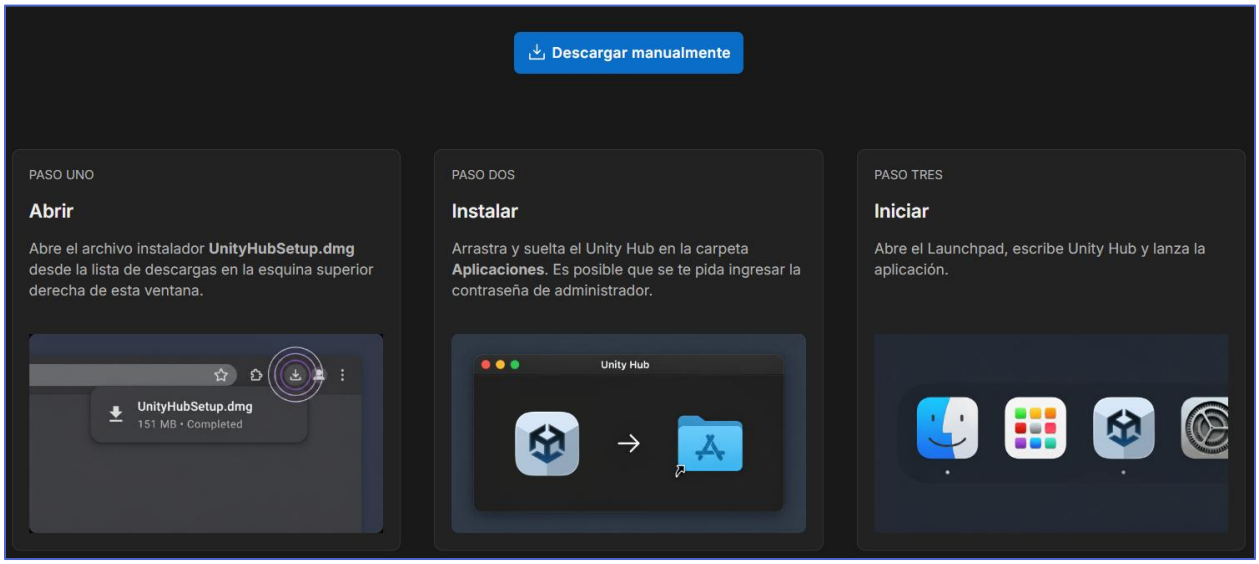
# 0 1

# PREPARANDO EL TERRENO: LA BATALLA CON EL ENTORNO DE DESARROLLO

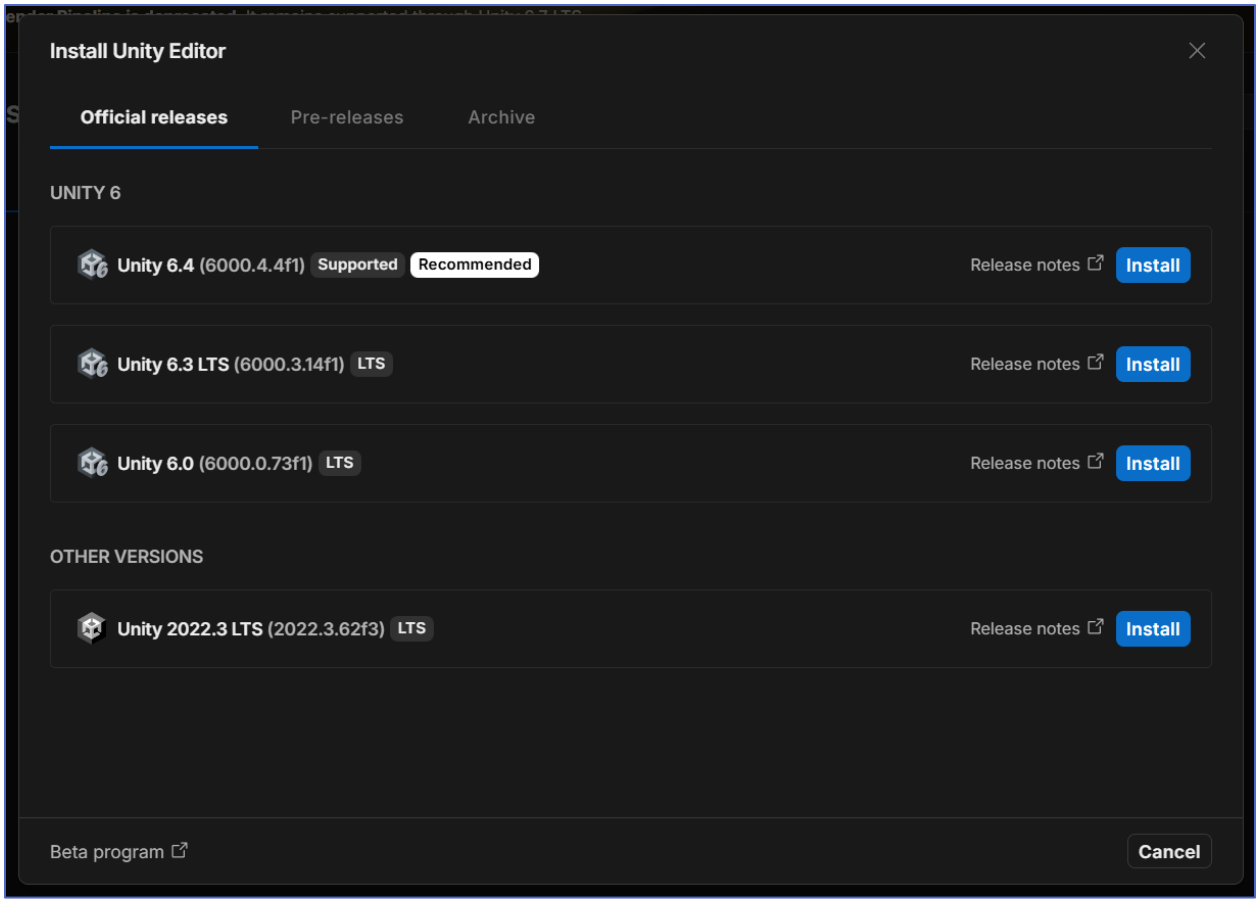
Antes de ponernos a medir terrenos, el primer reto fue montar el propio entorno. Unity no es el típico programa de "siguiente, siguiente, instalar", así que nos tocó investigar un poco cómo gestiona los proyectos.

## 1.1. El Unity Hub y la decisión de la versión

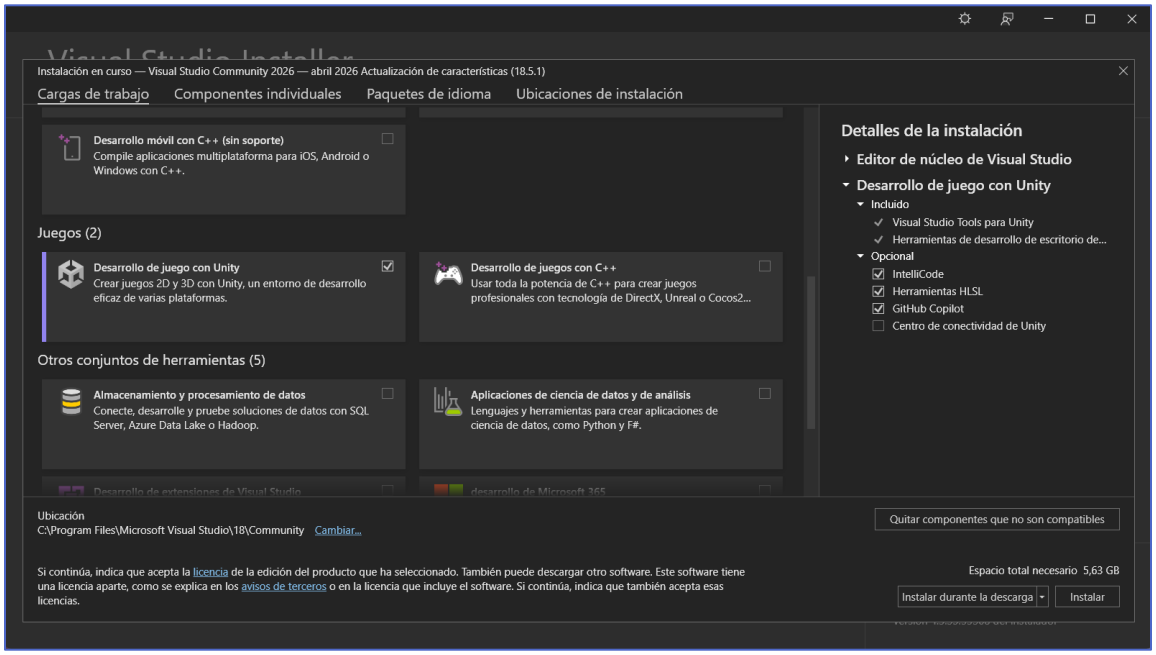
Vimos que la forma estándar de trabajar ahora es usar un **gestor intermedio** llamado **Unity Hub**.



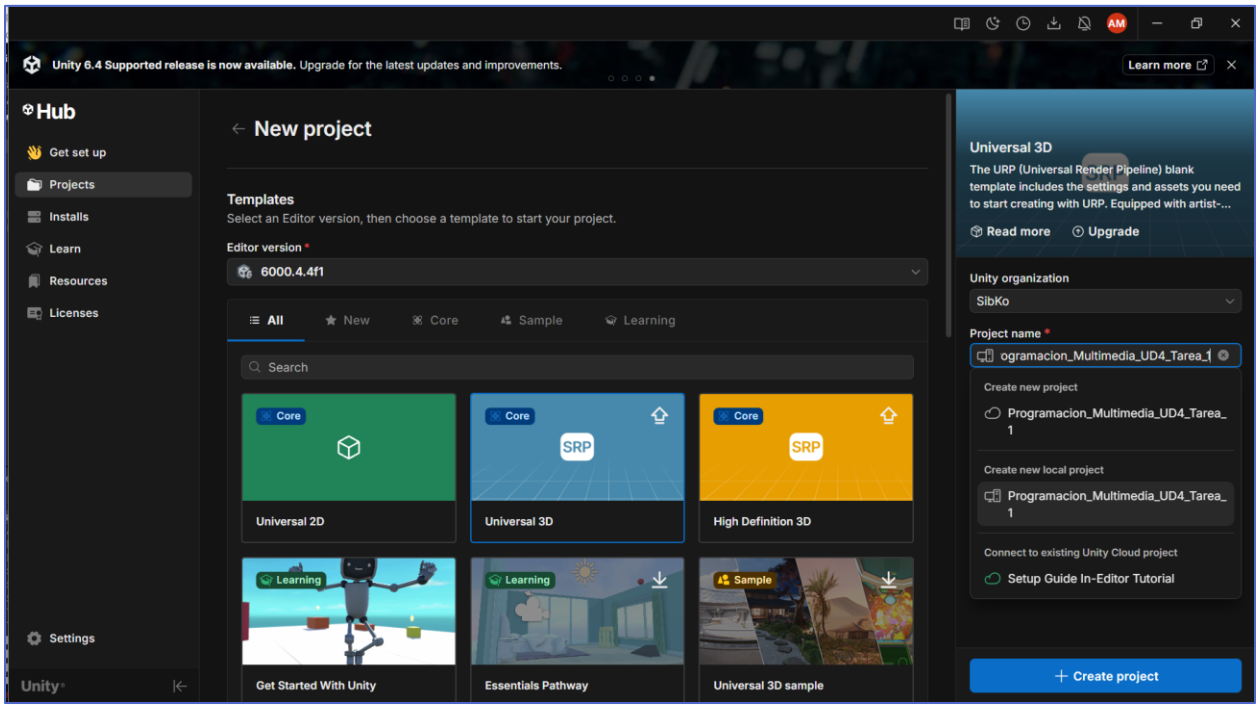

Nos registramos para sacar la **licencia** Personal de **estudiantes** y, al ir a instalar el editor, nos saltaron un montón de versiones distintas.



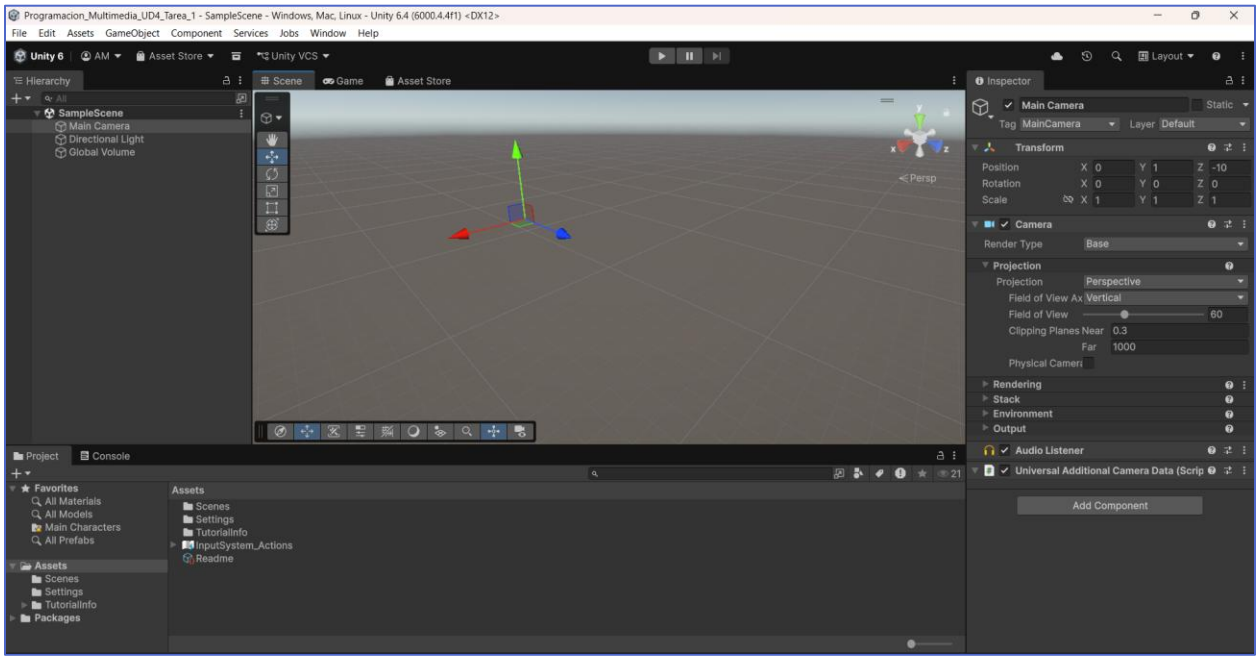

Decidimos ir a lo seguro e **instalar** la versión recomendada **Unity 6.4 (*Supported*)**. Cualquier desarrollador sabe que la estabilidad es lo primero; no queríamos usar una versión que nos rompiera los *scripts* o los materiales justo el día antes de la entrega por culpa de una actualización rara.



Una vez instalado todo, creamos un proyecto con la **plantilla "Core Universal 3D"**.




Esto nos dejó una **escena limpia** con la cámara principal y una luz direccional. Ya podíamos empezar.



# 02

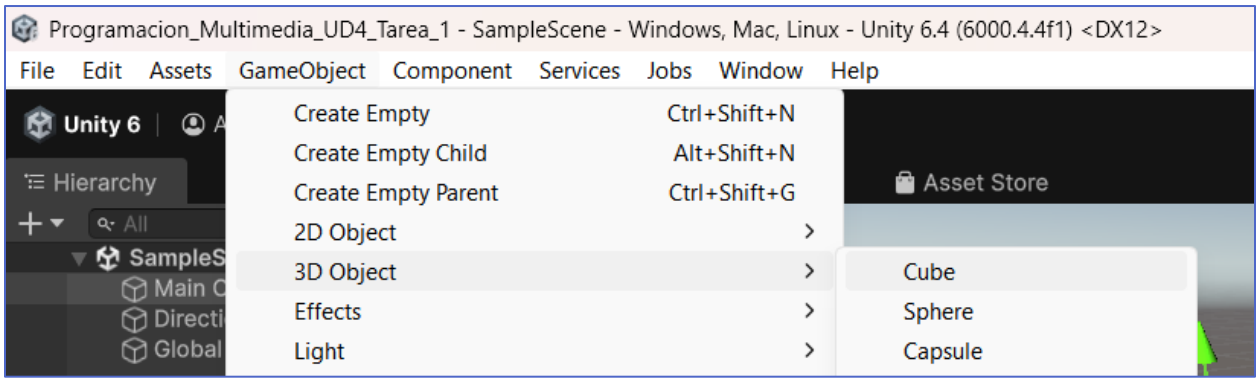
# EL DILEMA DEL MODELADO: ¿TERRAIN ENGINE O PRIMITIVA 3D?

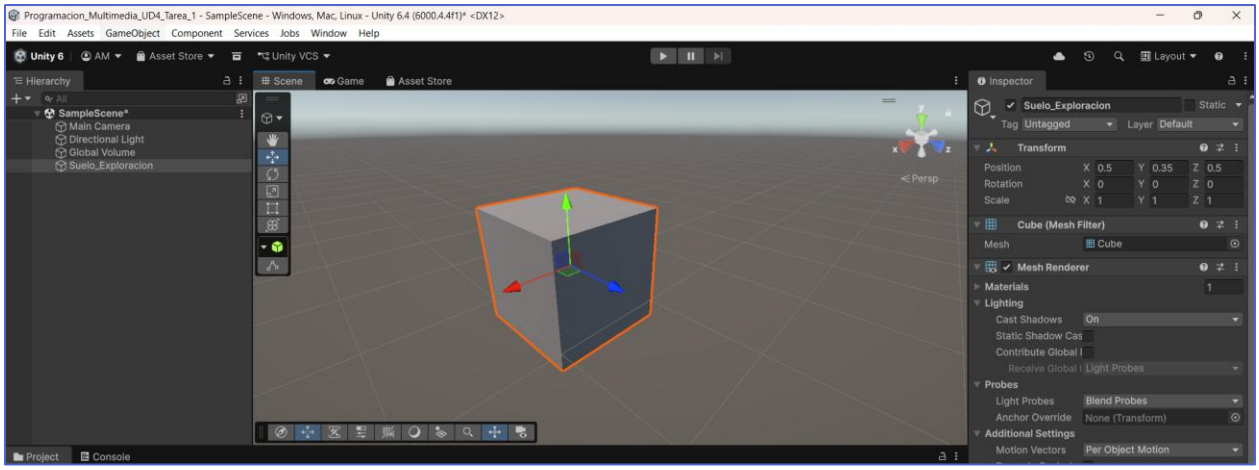
Aquí llegó el primer momento de pararse a pensar. La rúbrica pedía un objeto tipo **"terreno" de 30x10**. Nuestro primer instinto fue tirar por el camino fácil: GameObject > 3D Object > Terrain. El problema es que Unity te planta ahí por defecto un trozo de tierra brutal de 1000x1000 metros.

Recortar semejante monstruo para dejarlo en 30x10 era matar moscas a cañonazos. Ese componente está pensado para esculpir montañas, generar bosques enteros y gestionar el nivel de detalle (LOD) en juegos de mundo abierto. Usarlo para una plataforma tan pequeña nos pareció un desperdicio de recursos de la CPU totalmente injustificado. Por eso, **decidimos** tirar por lo eficiente y **construir el suelo a partir de un simple Cubo**.

## 2.1. Dimensionamiento a través del componente Transform

Creamos el **cubo** y lo llamamos **"Suelo_Exploracion"**.



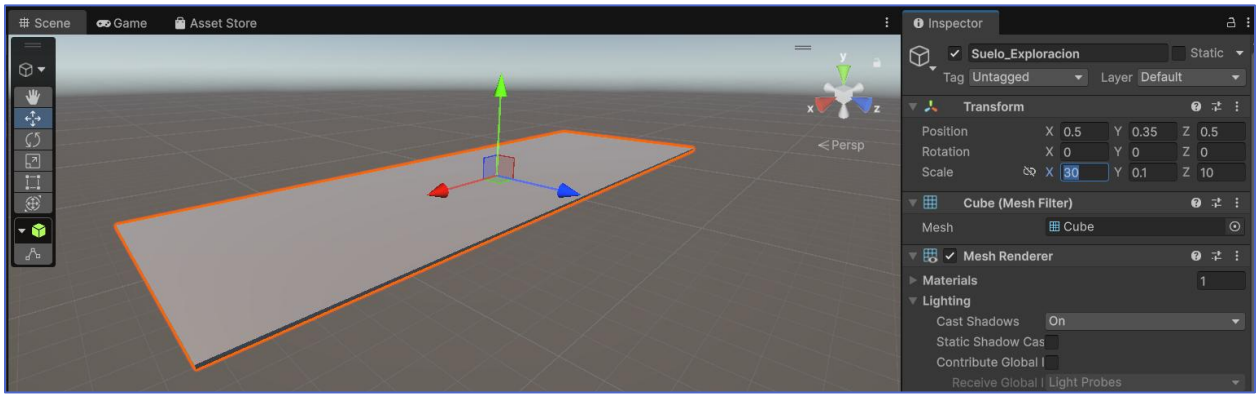



Luego nos fuimos al panel ***Inspector*** para ajustar el componente **Transform**, que básicamente es lo que controla dónde está el objeto y cuánto mide en el espacio. Como sabemos que la **escala 1 en Unity es igual a 1 metro en el mundo real**, tocamos los valores de **Scale** así:

- **Eje X (Anchura):** Le pusimos 30.

- **Eje Y (Grosor):** Si lo dejábamos en 1, iba a parecer un muro caído en lugar de un suelo. Lo bajamos a 0.1 para que quedara plano pero tuviera algo de cuerpo. (Descartamos usar un Plane normal porque los planos no tienen volumen real y por debajo son invisibles por el *backface culling*, lo que a veces da fallos raros con las físicas si algo se cae).

- **Eje Z (Profundidad):** Le pusimos 10.




Con esto clavamos las medidas exactas que se pedían y, de paso, evitamos sobrecargar el motor gráfico usando geometría innecesaria.

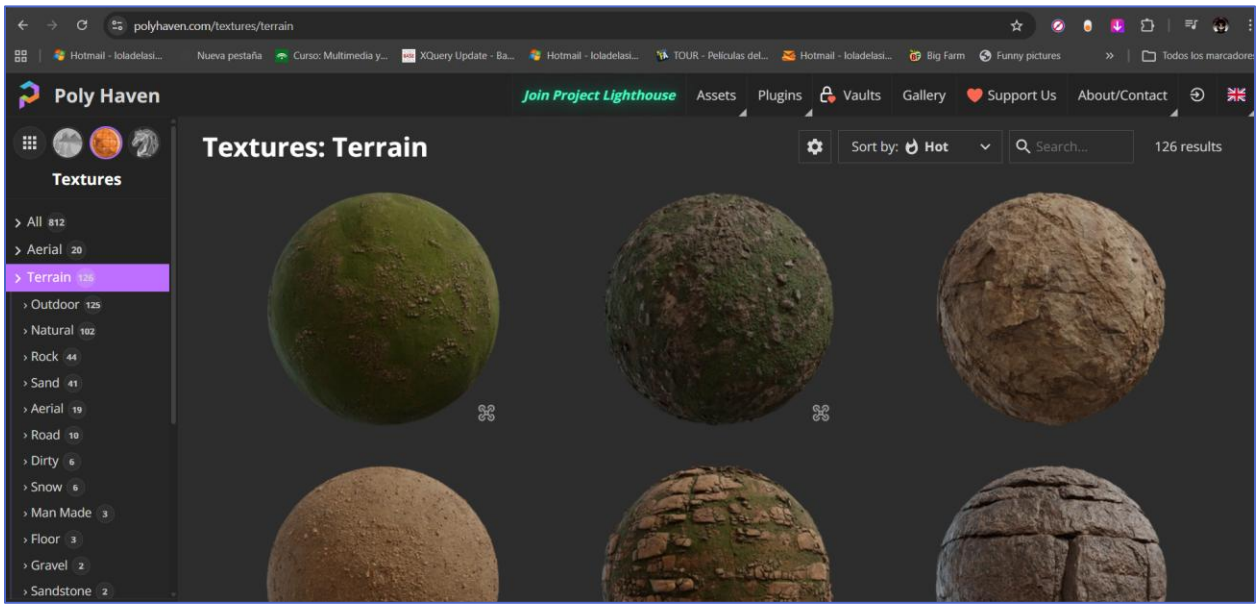
# 0 3

# LA TEXTURIZACIÓN: MÁS ALLÁ DEL SIMPLE COLOR

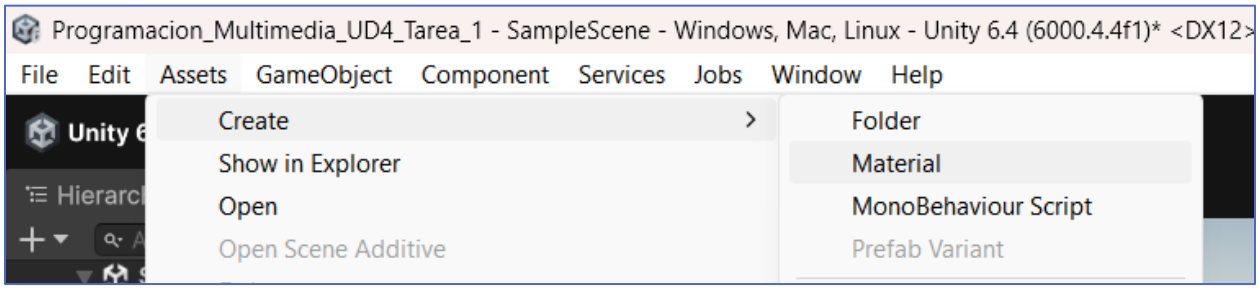
Teníamos que darle aspecto al terreno. Para no entregar la típica plataforma con textura de césped de baja calidad, decidimos darle un rollo de exploración espacial, buscando un **acabado tipo roca volcánica o superficie de Marte**.

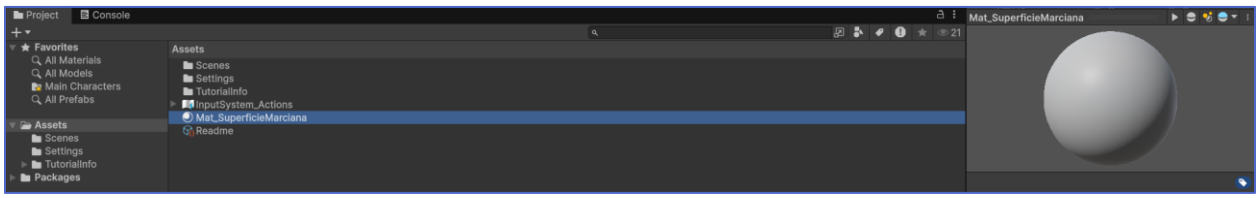
## 3.1. La diferencia fundamental entre Textura y Material

Nos bajamos un set de **texturas 2K de Poly Haven** que fuesen ***seamless*** (para que no se noten los cortes al repetirse).



El primer fallo de novatos fue intentar arrastrar la imagen JPG directamente al cubo. Obviamente, Unity no funciona así. Primero tuvimos que **crear** un **Material** nuevo (**Create > Material**) y llamarlo **Mat_SuperficieMarciana**.

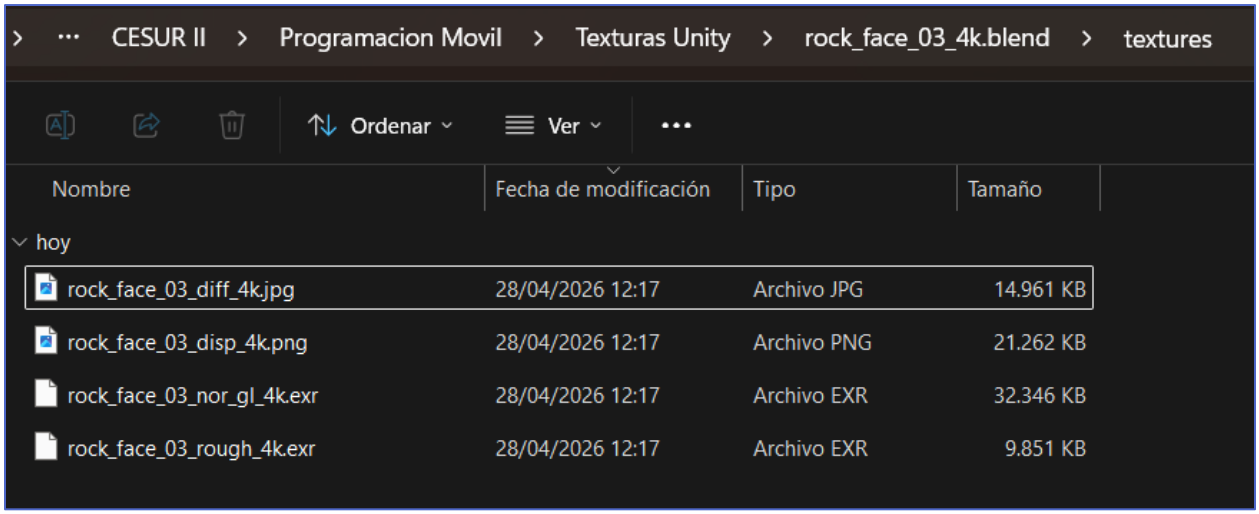


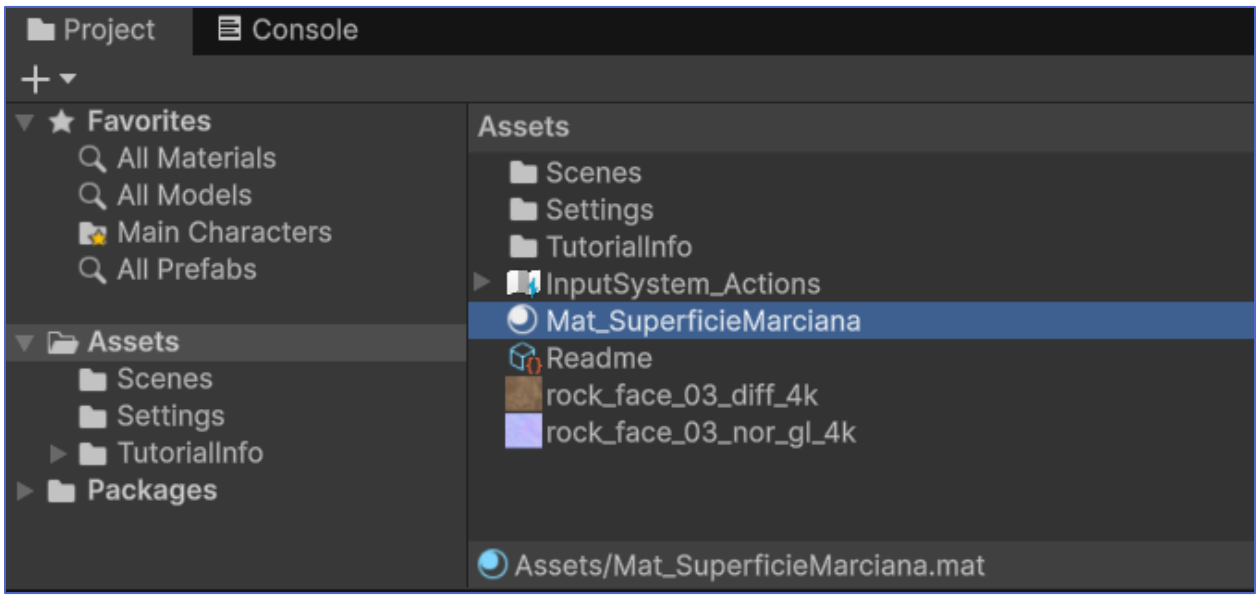

El material es como la capa intermedia que le dice a la gráfica cómo interpretar la imagen cuando le da la luz. Lo dejamos configurado con el ***Universal Render Pipeline*** con la opción **Lit** activada (iluminación) para que el resultado fuera realista.

## 3.2. Aplicando Mapas de Renderizado Físico (PBR)

Para que el suelo tuviera buen aspecto, no servía solo con poner una foto roja. En el *Inspector* del material mapeamos dos cosas:



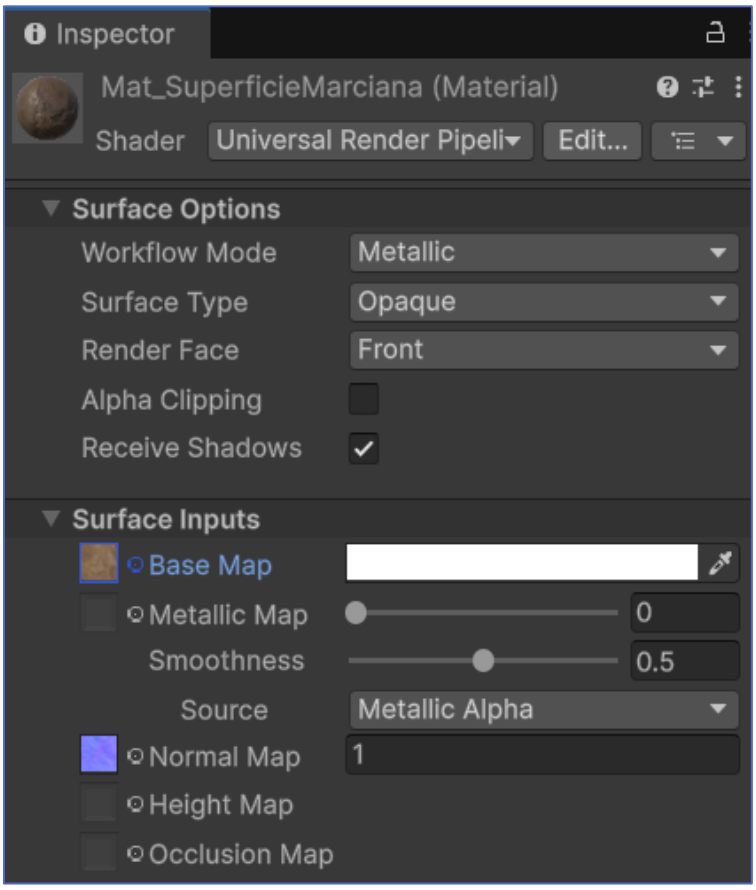

Arrastramos los archivos que vamos a usar a la carpeta de Assets del proyecto. En Unity no se puede arrastrar desde Windows directamente a un proyecto.



- En el canal **Base Map**, metimos la imagen base con el color del polvo y la roca.
  (*usamos los archivos de texturas con **diff** en el nombre*)

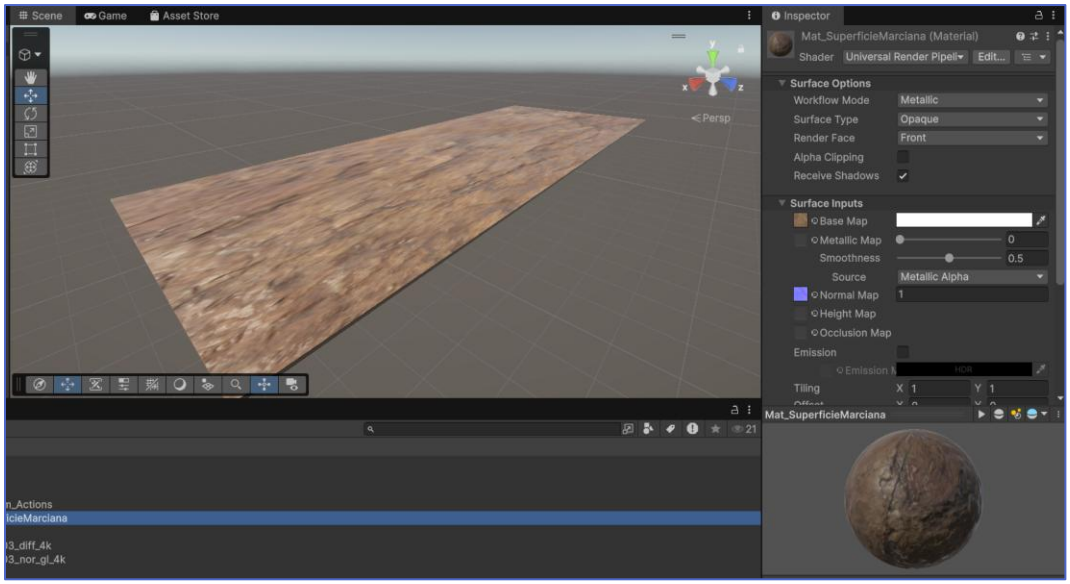
- En el **Normal Map**, pusimos esa textura azulada/violácea que trae el set y que marca el relieve.
  (*usamos los archivos de texturas con **nor** o normal en el nombre*)

  Engaña a la luz para que cree sombras en los huecos de la imagen. Nos da una sensación de profundidad brutal sin haber añadido ni un solo polígono de más al cubo.



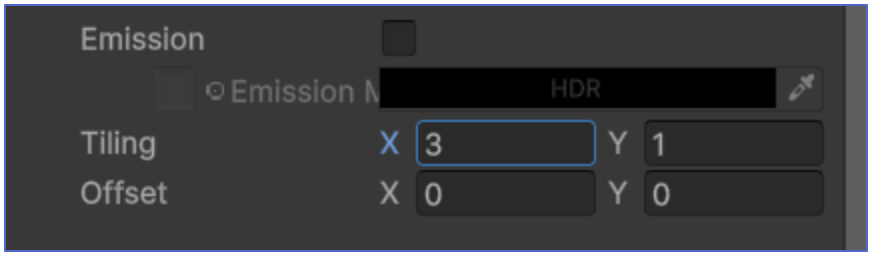
## 3.3. El problema matemático del "Tiling" (Embaldosado)

El susto vino cuando arrastramos el material ya terminado al cubo de 30x10. Se veía fatal. Las texturas estaban estiradísimas y las piedras parecían simples líneas borrosas cruzando el mapa.



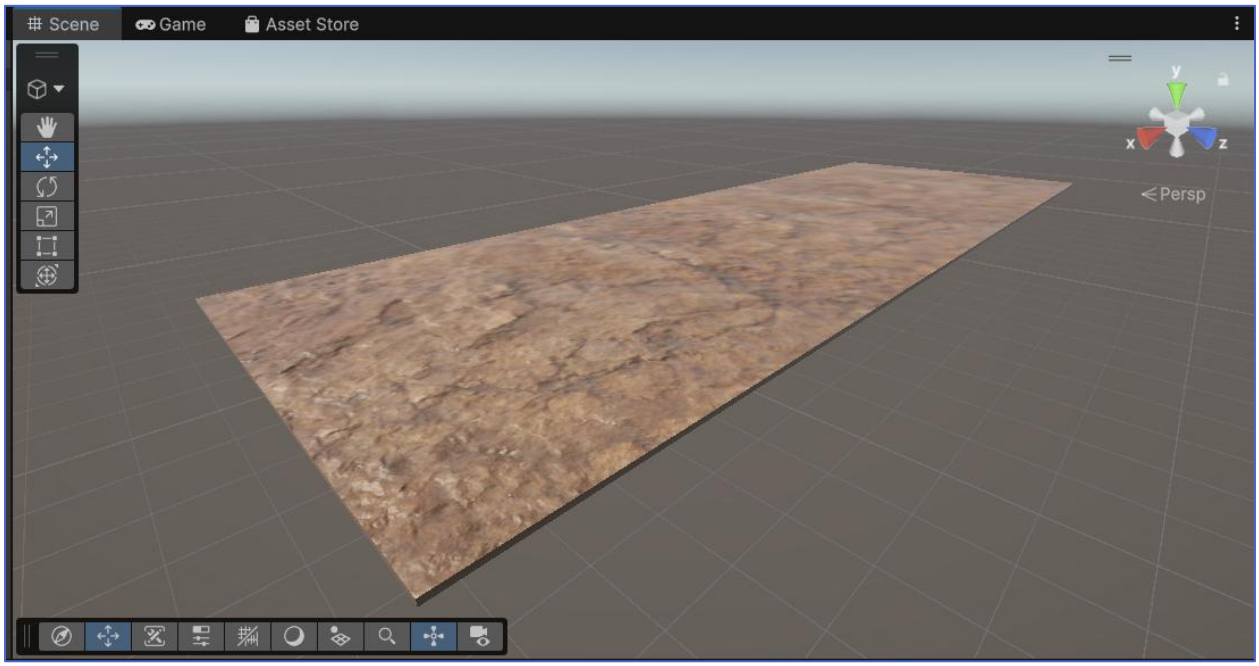

La imagen original que habíamos descargado era completamente cuadrada (proporción 1:1), pero nuestro suelo era rectangular y muy alargado (30 de ancho por 10 de fondo, un ratio de 3:1). Unity cogía ese cuadrado y lo estiraba a lo bestia para cubrir los 30 metros. Arreglarlo fue cuestión de tocar el parámetro de repetición. Entramos al **material** y fuimos a **Main Maps > Tiling**. Le metimos estos valores:

- **Tiling X = 3** (Para que repita la loseta de la textura 3 veces a lo ancho).
- **Tiling Y = 1** (Lo dejamos igual porque el fondo era 10).

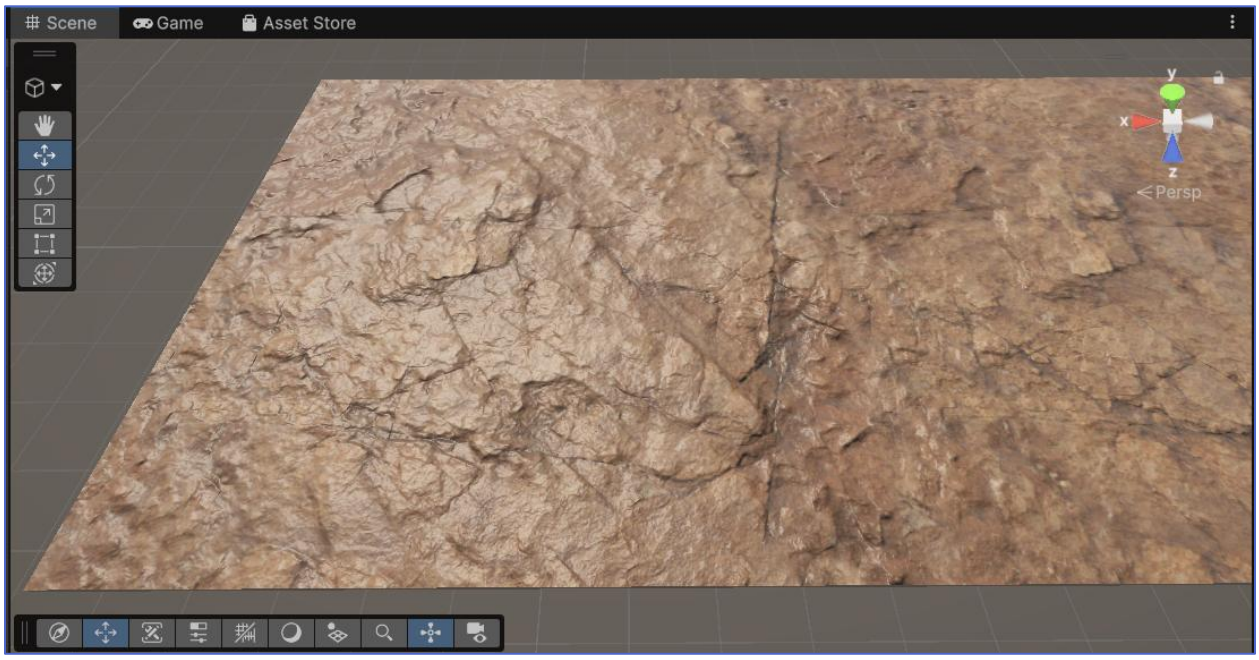



Fue mano de santo. Al dividir visualmente la textura en tres bloques a lo ancho, la proporción se compensó perfectamente. Las rocas recuperaron su forma redonda original y cubrían todo el terreno sin que se notara la deformación.

Resultado con la proporción de la textura correcta.




Hacemos zoom para apreciar los detalles de la calidad de la textura



# 0 4
# C O N C L U S I O N E S

Terminar este caso práctico nos deja con un muy buen sabor de boca. Ya no es solo haber conseguido hacer el dichoso terreno de 30x10 metros virtuales, sino todo lo que hemos asimilado "trasteando" por debajo. Nos ha quedado clarísimo que en el mundo del 3D el componente *Transform* es el rey absoluto; si no controlas bien la escala desde el principio, luego las luces, las físicas y el renderizado hacen cosas rarísimas. Además, uno de los mayores logros ha sido entender por fin la enorme diferencia que hay entre arrastrar una simple textura y configurar un material de verdad. Al usar el *shader* moderno *Lit* de URP y corregir el *Tiling* a base de matemáticas (poniendo la repetición a 3 en el eje X y a 1 en el eje Y), logramos que nuestro suelo volcánico se viera totalmente nítido y sin estirarse, que suele ser el típico fallo cantoso de novato.

Mirándolo con perspectiva, creemos que este trabajo nos ha servido mucho para soltarnos a la hora de sincronizar distintas aplicaciones. No nos hemos quedado encerrados en el editor de Unity: hemos gestionado las instalaciones con Unity Hub, hemos preparado el entorno de código con Visual Studio, y hemos buceado por repositorios profesionales como Poly Haven para traernos mapas de normales y de *albedo* de alta calidad. Saber importar todo esto y lidiar con la compatibilidad de los archivos es lo que al final te da la habilidad de montar un proyecto multimedia que tenga buena pinta. Sinceramente, ahora nos sentimos mucho más preparados para los siguientes retos de la asignatura, con la tranquilidad de que ya sabemos cómo montar las bases para los futuros juegos de móviles que tendremos que desarrollar.

# 0 5

# R E F E R E N C I A S


https://docs.unity3d.com/Manual/index.html

https://docs.unity3d.com/Manual/class-Transform.html

https://docs.unity3d.com/Manual/StandardShaderMaterialParameters.html

https://learn.microsoft.com/es-es/visualstudio/gamedev/unity/get-started/getting-started-with-visual-studio-tools-for-unity?pivots=windows

https://polyhaven.com/textures

https://learn.unity.com/

https://learn.unity.com/search?q=Materials+and+Textures

**Unity 6.4: VFX Graph Snow Deep Dive: Texture Indexing, Turbulence, Kill Planes Intermediate Tutorial**
https://www.youtube.com/watch?v=hUsxqGz5LR8

**125 - Fix Pink Textures in Unity 6 for custom assets**

https://www.youtube.com/watch?v=Y_fkAudFxeE

**UNITY 6 TUTORIAL PART 1 - LEARN THE BASICS - HOW TO MAKE A GAME FOR BEGINNERS**

https://www.youtube.com/watch?v=HwI90YLqMaY&list=PLZ1b66Z1KFKhO7R6Q588cdWxdnVxpPmA8